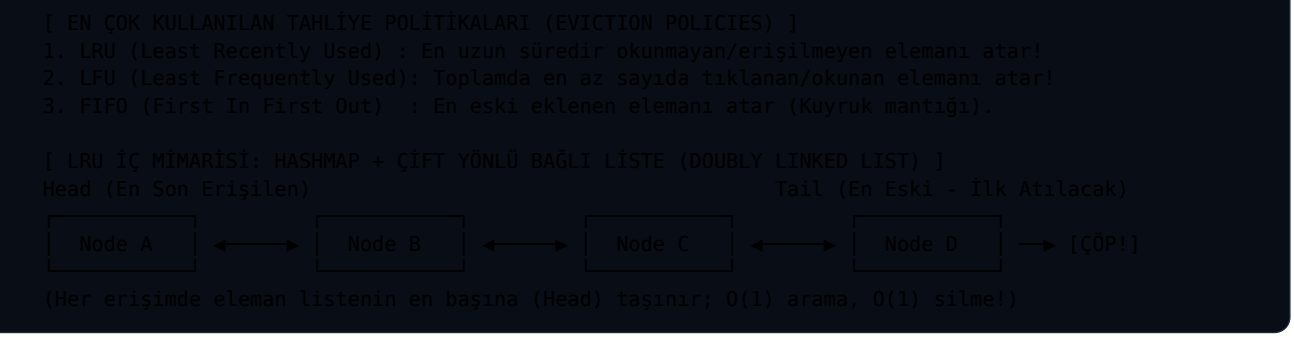
15

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Tahliye Algoritmaları: LRU, LFU, FIFO ve Java 25 LRU Cache

Cache Eviction Algorithms: LRU, LFU, FIFO & Java 25 LRU Cache

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Çalışma masanızın üzerinde en fazla 5 kitap koyabileceğiniz bir alan olduğunu hayal edin (Önbellek boyutu). 6. kitabı koymak için birini rafa kaldırmalısınız. LRU (Least Recently Used), 'En uzun süredir kapağını bile açmadığım kitabı kaldırayım' diyen mantıktır. LFU (Least Frequently Used) ise 'Sene boyunca toplamda en az okuduğum kitabı kaldırayım' der. Masadaki kitaplardan birini her okuduğunuzda onu masanın en önüne (Head) çekersiniz; masanın en arkasında kalan kitap (Tail) ise ilk atılacak kurbandır!

🇬🇧 ENGLISH (ELI5)

Imagine your desk only fits 5 textbooks. To place a 6th book, you must return one to the library shelf. LRU (Least Recently Used) evicts the book you haven't touched in the longest time. LFU (Least Frequently Used) counts tally marks and evicts the book with the lowest total read count. Whenever you consult a book, you slide it to the front of your desk. The book lingering neglected at the far corner is the first victim evicted on overflow!

🔍 MAANG Seviyesi Teknik Derinlik — 🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG mülakatlarında en çok kodlatılan veri yapısı LRU Cache'tir. Bir LRU Cache iki temel işlem için $O(1)$ zaman karmaşıklığı sunmalıdır: 1) `get(key)`: Anahtarı bulmalı ve düğümü en başa (Head) taşımalıdır. 2) `put(key, value)`: Yeni düğümü başa eklemeli, kapasite aşıldığında sondaki (Tail) düğümü silmelidir. Bu $O(1)$ garantisi HashMap ($O(1)$ anahtar araması) ile Doubly Linked List'in ($O(1)$ düğüm çıkarma ve ekleme) birleşimiyle sağlanır. LFU ise frekans sayacı tutar; ancak ani popülerlik dalgalanmalarına (flash crowds) uyum sağlaması zordur.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The LRU Cache is a premier FAANG coding interview staple. It guarantees $O(1)$ time complexity for both `get` and `put` operations. This is engineered by coupling a Hash Table (for $O(1)$ pointer lookups) with a Doubly Linked List (for $O(1)$ node extractions and insertions at Head and Tail boundaries). When a node is touched, it splices out of its current position and prepends to the Head. On capacity overflow, the Tail node drops. LFU maintains frequency buckets but suffers from historical inertia where old items refuse to evict despite going cold.

⚡ Modern Java 25 Uygulaması (Loom & Records) / ⚡ Java 25 Implementation

```java
// Java 25: O(1) Zaman Karmaşıklığına Sahip Gerçek LRU Cache Implementasyonu
import java.util.*;

public class LruCache<K, V> {
    private static class Node<K, V> {
        K key; V value;
        Node<K, V> prev, next;
        Node(K k, V v) { this.key = k; this.value = v; }
    }

    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final Node<K, V> head = new Node<>(null, null);
    private final Node<K, V> tail = new Node<>(null, null);

    public LruCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public synchronized V get(K key) {
        Node<K, V> node = map.get(key);
        if (node == null) return null;
        moveToHead(node);
        return node.value;
    }

    public synchronized void put(K key, V value) {
        Node<K, V> node = map.get(key);
        if (node != null) {
            node.value = value;
            moveToHead(node);
        } else {
            if (map.size() >= capacity) {
                map.remove(tail.prev.key);
                removeNode(tail.prev);
            }
            Node<K, V> newNode = new Node<>(key, value);
            map.put(key, newNode);
            addToHead(newNode);
        }
    }

    private void removeNode(Node<K, V> node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void addToHead(Node<K, V> node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    private void moveToHead(Node<K, V> node) {
        removeNode(node);
        addToHead(node);
    }
}

void main() {
    var lru = new LruCache<String, Integer>(2);
    lru.put("A", 1);
    lru.put("B", 2);
    IO.println("Get A: " + lru.get("A")); // A başa geçti, B arkaya düştü
    lru.put("C", 3);                      // Kapasite taştı -> B atıldı!
    IO.println("Get B (Tahliye Edildi): " + lru.get("B")); // null döner!
    IO.println("Get C: " + lru.get("C"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Java'nın `LinkedHashMap(capacity, 0.75f, true)` sınıfı LRU mantığını hazır sunar. Ancak MAANG mülakatında hazır kütüphane çağırmak yerine HashMap + Doubly Linked List veri yapısını kendi elinizle çizip açıklamanız beklenir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

While Java provides `LinkedHashMap(..., accessOrder=true)`, invoking it without explaining the underlying Doubly Linked List and Hash Map duality will disqualify your algorithmic rigor in Staff rounds.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LRU en son erişilen veriyi korur ve zamansal yerellik (temporal locality) için mükemmeldir.
- O(1) süre garantisi HashMap ile Çift Yönlü Bağlı Liste kombinasyonuyla elde edilir.
- Redis varsayılan olarak yaklaşık (approximate) LRU algoritması kullanarak bellekten tasarruf eder.

🎯 Key Takeaways for the Interview

- LRU leverages temporal locality, keeping recently accessed objects warm.
- Combining a Hash Map with a Doubly Linked List yields strict O(1) lookups and evictions.
- Production Redis uses probabilistic sampling to approximate LRU with zero pointer overhead.